OS-ORANGE-ASSIGNMENT-2

NAME:VIREN JAMES FERNANDES

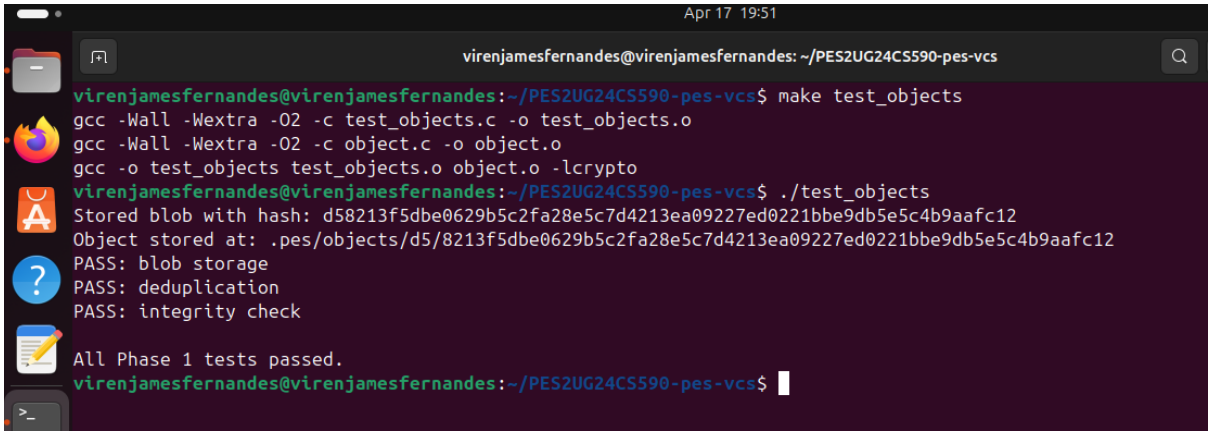
SRN:PES2UG24CS590

SEM:4

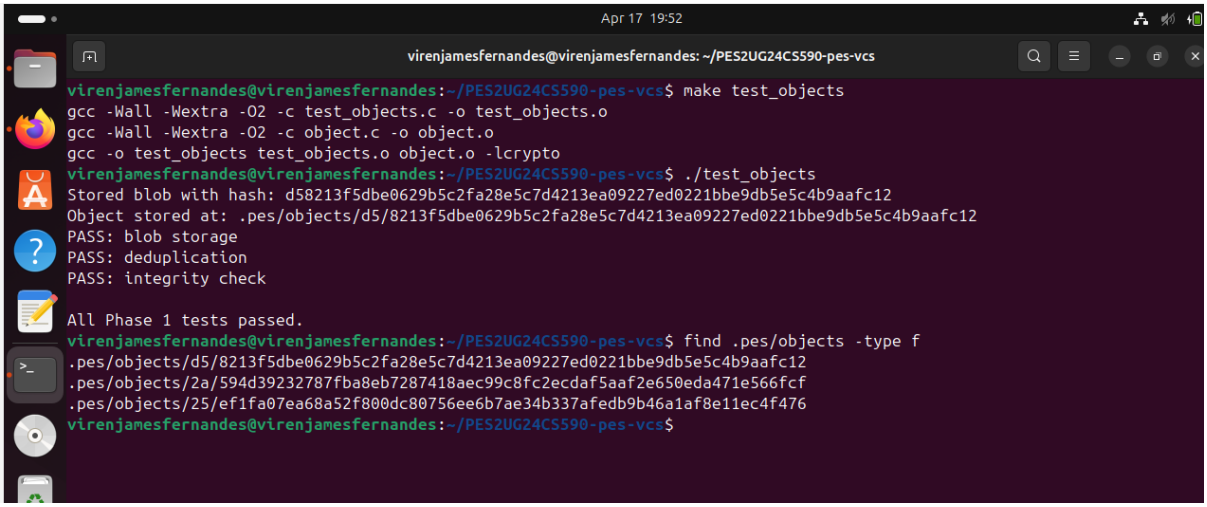
SEC:J

Phase 1

1.a Screenshot:

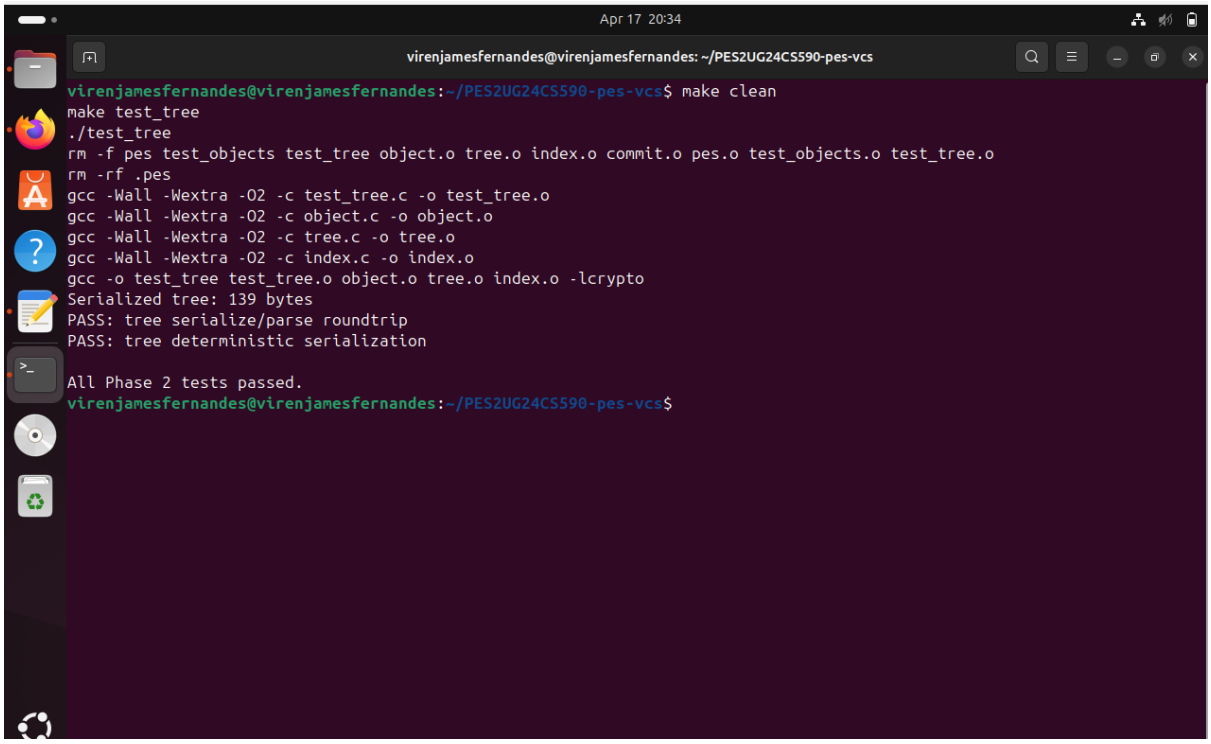


1.b Screenshot:

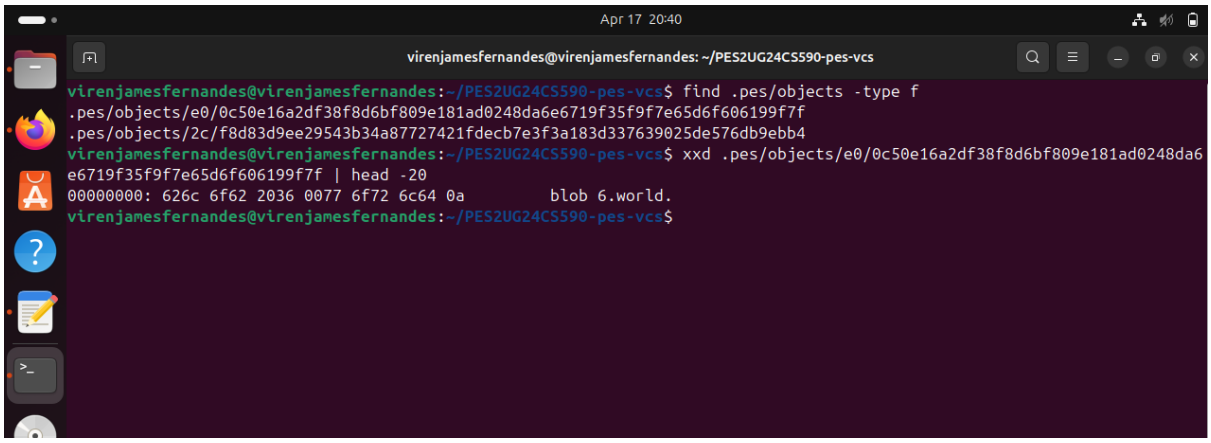


Phase 2:

2.a Screenshot

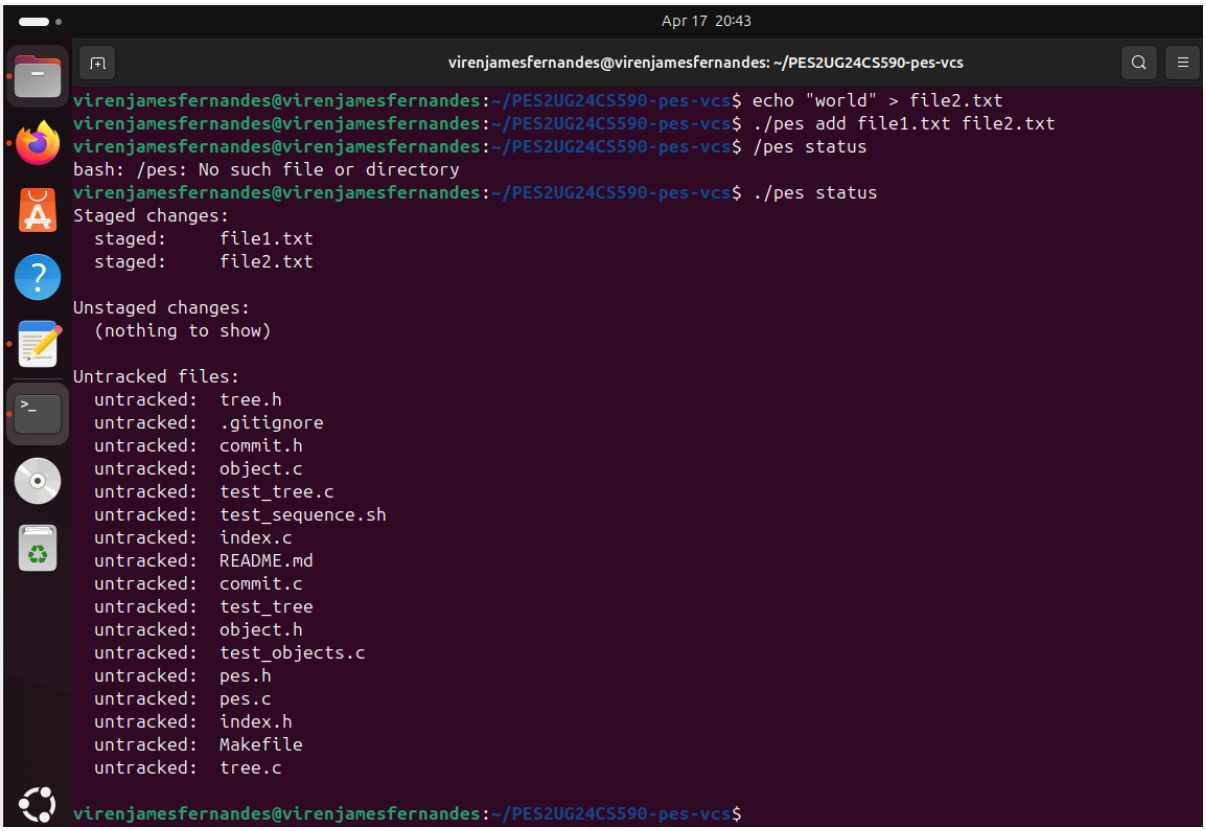


2.b Screenshot:

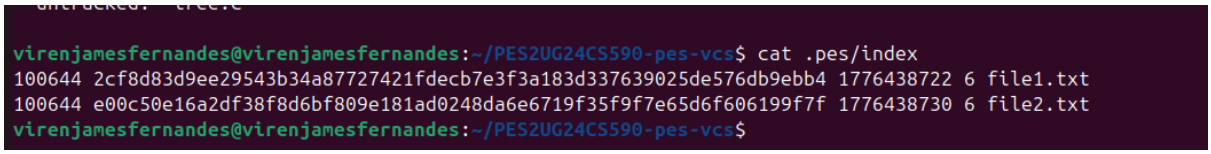


Phase 3:

Screenshot 3.a:



Screenshot 3.b:



Phase 4:

Screenshot 4.a:

```
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
Initialized empty PES repository in .pes/
Committed: 64b76ecc95ac... Initial commit
Committed: d9cbba67ebe2... Add world
Committed: f2d8d0b0f5c6... Add farewell
commit f2d8d0b0f5c6e6f18c48677fcc75660a573baaa36de8c436b87671c8a85c20fa
Author: Viren Fernandes <PES2UG24CS590>
Date:   1776438883

    Add farewell

commit d9cbba67ebe2ba1700860afe6f6faaf04e8b84182b4ea20e1c3b0c6e9f526dd4
Author: Viren Fernandes <PES2UG24CS590>
Date:   1776438883

    Add world

commit 64b76ecc95ac2f942ecb3f7d9e52c7d0533fd913ae3fdd0c1b8e7828550bee24
Author: Viren Fernandes <PES2UG24CS590>
Date:   1776438883
```

Screenshot 4.b:

```
Apr 17  20:45
virenjamesfernandes@virenjamesfernandes: ~/PES2UG24CS590-pes-vcs
virenjamesfernandes@virenjamesfernandes:~/PES2UG24CS590-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/64/b76ecc95ac2f942ecb3f7d9e52c7d0533fd913ae3fdd0c1b8e7828550bee24
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/85/e367b76c2417b47a95f46208835ad8700dd58335171c94a497974be6191900
.pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
.pes/objects/d9/cbba67ebe2ba1700860afe6f6faaf04e8b84182b4ea20e1c3b0c6e9f526dd4
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f1/0239b620372163c2dd06ac33189180c55edf1c56b15990a1cf44e00b1b0e59
.pes/objects/f2/d8d0b0f5c6e6f18c48677fcc75660a573baaa36de8c436b87671c8a85c20fa
.pes/refs/heads/main
virenjamesfernandes@virenjamesfernandes:~/PES2UG24CS590-pes-vcs$
```

Screenshot 4.c:

```
virenjamesfernandes@virenjamesfernandes:~/PES2UG24CS590-pes-vcs$ cat .pes/refs/heads/main
f2d8d0b0f5c6e6f18c48677fcc75660a573baaa36de8c436b87671c8a85c20fa
virenjamesfernandes@virenjamesfernandes:~/PES2UG24CS590-pes-vcs$
```

Screenshot 4.d:

```
virenjamesfernandes@virenjamesfernandes:~/PES2UG24CS590-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
virenjamesfernandes@virenjamesfernandes:~/PES2UG24CS590-pes-vcs$
```

Phase 5 and 6:

Branching and Checkout

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

ANS: To switch branches, you essentially need to move a pointer and synchronize the disk.

- **File Changes in .pes/:** * **HEAD:** This file must be updated to point to the new branch reference (e.g., ref: refs/heads/<branch>).
    - **The Index (Staging Area):** The index must be overwritten with the tree structure of the commit that <branch> points to.
- **Working Directory Changes:** The files in your actual folders must be updated to match the blobs and trees of the target commit. This involves deleting files that don't exist in the new branch and creating/modifying files that do.
- **The Complexity:** The difficulty lies in **safety**. If the working directory has "dirty" (unsaved) changes, a naive checkout would overwrite them, causing permanent data loss. You must also handle the transition efficiently so you don't re-write 10,000 files if only two actually changed between branches.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

ANS: To detect a "dirty working directory" conflict during a checkout using only the **Index** (staging area) and the **Object Store**, you must perform a three-way comparison between the current commit, the index, and the working directory.

Here is the logical flow to implement this detection:

1. The Three Reference Points

- **HEAD (Current Commit):** The tree the user is currently "on."
- **The Index:** The snapshot of what the user has staged for the next commit.
- **The Target (New Branch):** The tree the user is trying to switch to.

2. The Conflict Detection Algorithm

For every file in the repository, you compare the **File Hashes** stored in the Index against the hashes in the trees. The conflict is triggered if the following conditions are met:

- **Step A: Check for "Staged" or "Modified" changes.** If the hash of the file in the **Index** is different from the hash of the file in the **current HEAD**, it means the file is "dirty" (it has either been modified in the working directory or staged).
- **Step B: Check for Branch Divergence.** Compare the hash of that same file in the **current HEAD** with the hash in the **Target Branch**. If these hashes are different, the file has changed between the two branches.
- **Step C: The Refusal Trigger.** If the file is **dirty** (from Step A) **AND** the file **differs between branches** (from Step B), the checkout must refuse.

3. Why this logic is used

If a file is "dirty" but it is **identical** in both the current branch and the target branch, Git can safely keep your local changes and carry them over to the new branch. The conflict only occurs when Git would be forced to overwrite your uncommitted work with a different version of that same file from the new branch.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

ANS: In a **Detached HEAD** state, the .pes/HEAD file contains a raw 40-character commit hash (e.g., d282f...) rather than a symbolic reference to a branch (e.g., ref: refs/heads/main).

Here is what happens during commits and how to rescue them:

What happens when you commit?

When you commit in a detached state, the version control system behaves almost normally, but with one critical difference: **no branch pointer moves forward.**

1. **New Commit Creation:** A new commit object is created. Its "parent" is the hash that was previously in HEAD.
2. **HEAD Updates:** The HEAD file is updated to point to the hash of this new commit.
3. **Lack of Persistence:** Because no branch (like main or dev) points to this new commit, it is effectively "floating." As soon as you run pes checkout main, HEAD moves to the main branch, and there is no longer any named reference pointing to the commits you just made. They become **orphaned**.

How to recover those commits?

If a user accidentally switches away from a detached HEAD and "loses" their work, the commits aren't immediately deleted—they are just hidden. They can be recovered using the following methods:

- **Using the Reflog:** The reflog (reference log) is a local record that tracks every time HEAD changes position. A user can run pes reflog, find the hash of the last commit they made while detached, and then create a new branch from it: pes branch recovery-branch <lost-hash>
- **The Object Store:** If the reflog isn't available, a user could theoretically search the .pes/objects directory for the most recently created commit objects, though this is much more difficult.
- **Preventative Recovery:** If the user is *still* on the detached commit and realizes they want to keep it, they can simply run pes checkout -b <new-branch-name> to create a branch at the current location and "attach" the HEAD to it.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

ANS:

To maintain a clean repository, you implement a **Mark-and-Sweep** garbage collection algorithm. This process identifies which objects are "live" by tracing references from known entry points and deleting everything else.

1. The Algorithm: Mark-and-Sweep

The process is divided into two distinct phases:

Phase 1: The Mark Phase (Reachability Tracing)

- **Identify Roots:** Start with all "references" (the **roots**). These include all branch heads (.pes/refs/heads/*), tags, and the current HEAD.
- **Recursive Traversal:** For every root hash, look up the object in the store:
    - If it's a **Commit**: Add its hash to the "reachable" set, then recurse into its tree hash and its parent commit hashes.
    - If it's a **Tree**: Add its hash to the "reachable" set, then recurse into every blob and sub-tree listed within it.
    - If it's a **Blob**: Simply mark the hash as "reachable" (blobs have no outgoing links).
- **Termination:** The recursion stops when it encounters a hash that has already been marked as reachable.

Phase 2: The Sweep Phase (Deletion)

- Iterate through every file in the .pes/objects/ directory.
- For each file, check if its hash exists in your "reachable" set.
- **Action:** If the hash is **not** in the set, the object is unreachable and can be safely deleted.

2. Efficient Data Structure

To track "reachable" hashes efficiently, a **Bloom Filter** combined with a **Hash Set** (specifically a HashSet<String> or a bitset if hashes are mapped to integers) is the gold standard.

- **Why a Hash Set?** It provides $O(1)$ average time complexity for both insertions (marking) and lookups (checking during the sweep).
- **Optimization:** In very large systems, a **Bloom Filter** can act as a fast preliminary check to rule out unreachable objects before querying the main set, reducing memory overhead.

3. Estimation for 100,000 Commits

For a repository with 100,000 commits and 50 branches, the number of objects to visit depends on the uniqueness of the files, but we can estimate based on typical structure:

- **Commits:** You must visit all **100,000** commits because even if they aren't branch heads, they are reachable through the parent-link chain of the 50 branches.
- **Trees:** Every commit has at least one top-level tree. While many commits share trees (if no files changed), a "busy" repo might have **~100,000 to 150,000** unique trees.
- **Blobs:** This is the largest category. If each tree points to an average of 5-10 unique blobs (accounting for deduplication across commits), you are looking at **~500,000 to 1,000,000** blobs.

Total Estimate: You would likely need to visit and track approximately **700,000 to 1.2 million objects**. Because the algorithm only visits each unique hash *once*, the performance is bound by the total number of unique objects in the history, not the number of branches.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

ANS: Running garbage collection (GC) simultaneously with a commit operation is dangerous because it creates a **race condition** between the process creating new data and the process deleting "unused" data. Since the object store is a shared resource, poor timing can lead to permanent repository corruption.

1. The Race Condition: "The Vanishing Blob"

Imagine two processes running at the exact same time: **Process A (Commit)** and **Process B (GC)**.

1. **Process A (Commit)** starts. It hashes a new file, creates a **Blob Object**, and writes it to the disk.
2. **Process B (GC)** starts its "Mark" phase. It looks at all existing branches to see what is reachable.
3. **The Gap:** Because Process A is still running, it hasn't updated the branch or HEAD yet. To the GC, the new Blob looks like "garbage" because nothing points to it.
4. **Process B (GC)** proceeds to the "Sweep" phase and **deletes the new Blob**.
5. **Process A (Commit)** finally tries to finish by creating a **Tree Object** that references the Blob's hash.
6. **Result:** The commit is finalized, but it points to a hash that no longer exists on disk. The repository is now corrupted (a "broken link" error)

2. How Git Avoids This: The Grace Period

Git’s real garbage collection (git gc) does not simply delete every unreachable object it finds. It uses two primary safety mechanisms:

The 14-Day Grace Period

By default, Git will only prune (delete) unreachable objects that are **older than two weeks**.

- When the GC identifies an unreachable object, it checks the file's **mtime** (modification time).
- If the object was created recently (e.g., within the last few minutes by a concurrent commit), the GC ignores it.
- This provides a massive safety buffer for any "in-flight" operations.

The gc.pruneExpire Setting

Users can configure this window, but the default is designed to be longer than any reasonable commit or rebase operation would ever take to complete.

Atomic Renames and Locking

For metadata like branch updates, Git uses **index.lock** files. While this doesn't prevent object deletion directly, it ensures that the "roots" of the reachability graph aren't shifting while the GC is trying to map them out.